



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



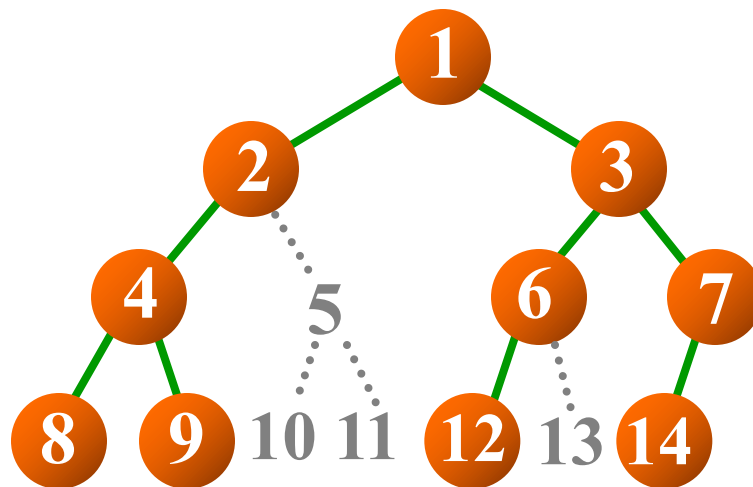
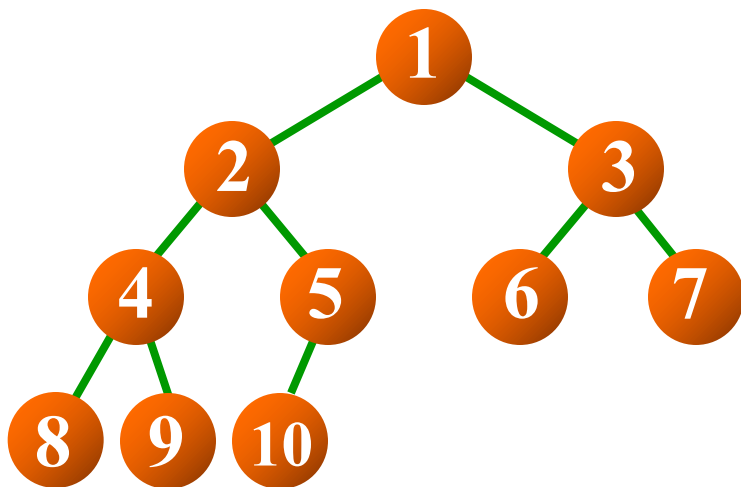
第五章 树与二叉树



- 树和森林的概念
- 二叉树
- 二叉树遍历
- 线索化二叉树
- 树与森林
- 堆
- Huffman树



5.3 二叉树的顺序表示

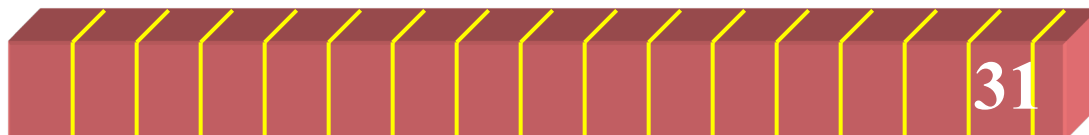


完全二叉树的顺序表示

一般二叉树的顺序表示



A diagram showing a sequence of numbers 1, 3, 7, 15, 31 in orange circles connected by a green line, representing a binary tree structure.





二叉树的数组表示的优点包括：

- 数组存储在连续的内存空间中，对缓存友好，访问与遍历速度较快。
- 不需要存储指针，比较节省空间。
- 允许随机访问节点。

然而，数组表示也具有有一些局限性：

- 数组存储需要连续内存空间，因此不适合存储数据量过大的树。
- 增删节点需要通过数组插入与删除操作实现，效率较低。
- 当二叉树中存在大量 None 时，数组中包含的节点数据比重较低，空间利用率较低。



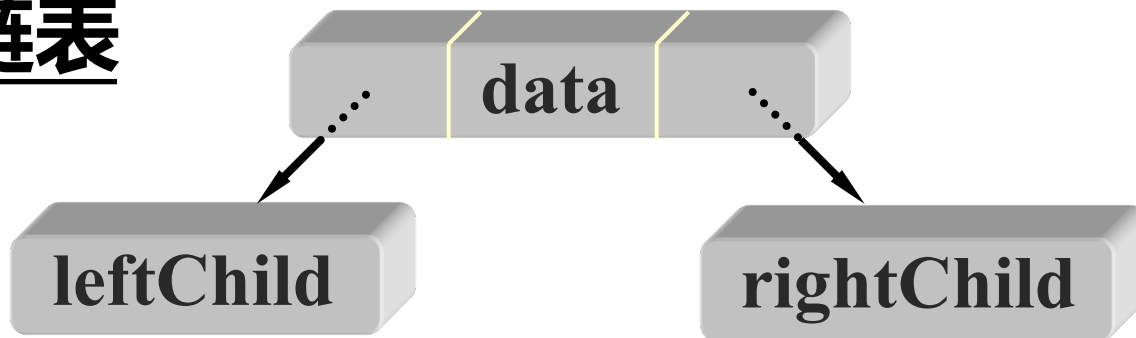
二叉树的链表表示（二叉链表）



- 二叉树结点定义：每个结点有3个成员，data域存储结点数据，leftChild和rightChild分别存放指向左子女和右子女的指针。



二叉链表





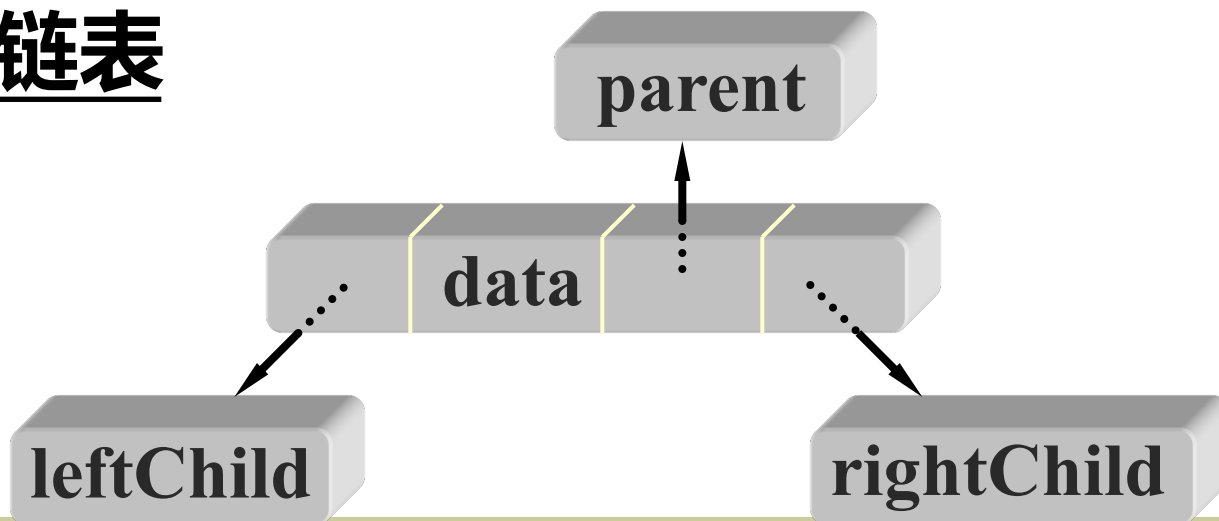
二叉树的链表表示（三叉链表）

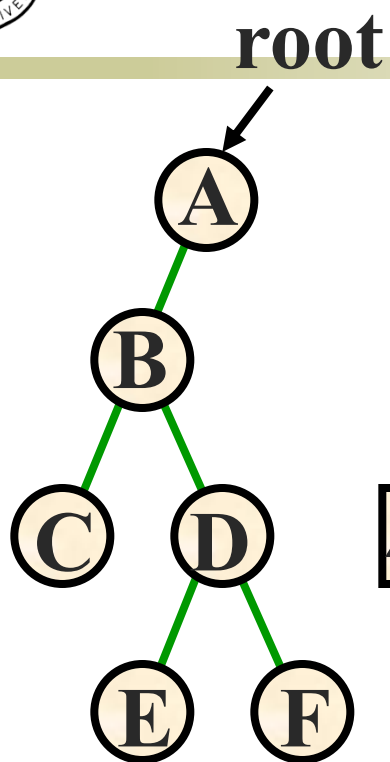


- 每个结点增加一个指向双亲的指针parent, 使得查找双亲也很方便。

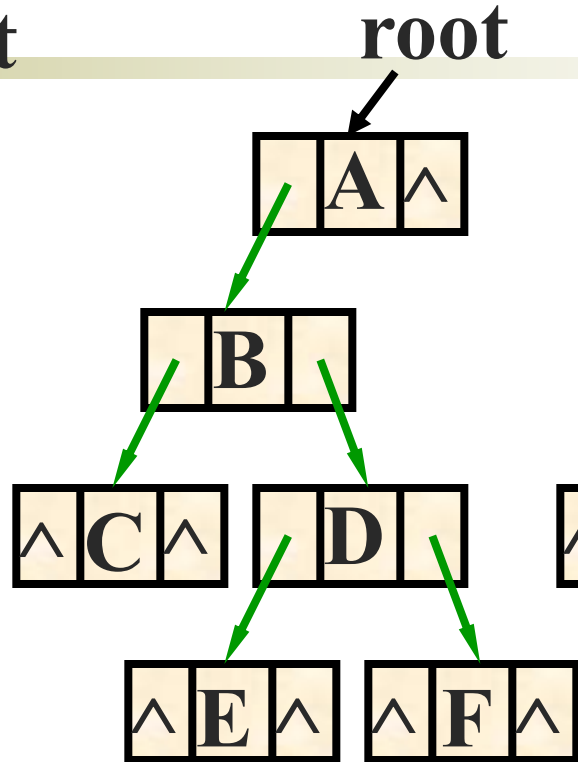


三叉链表

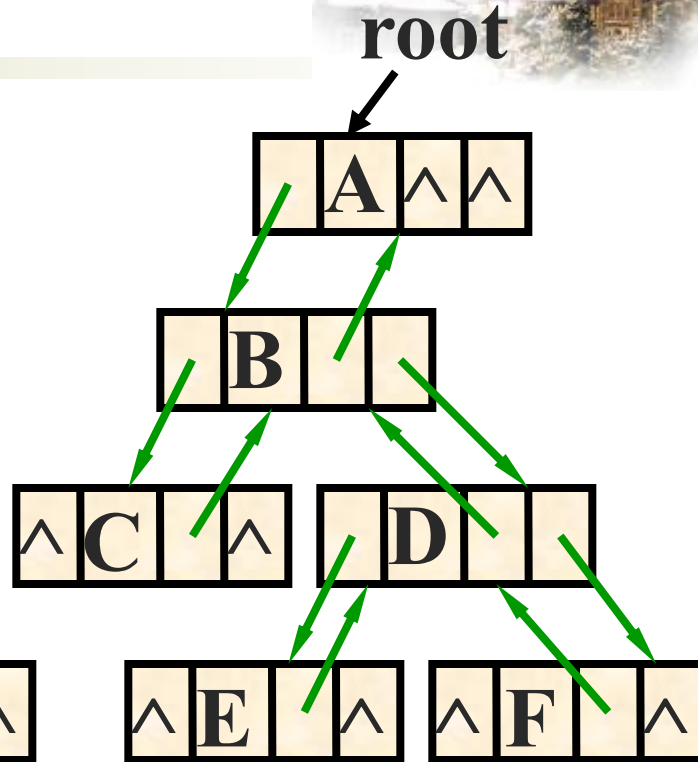




二叉树

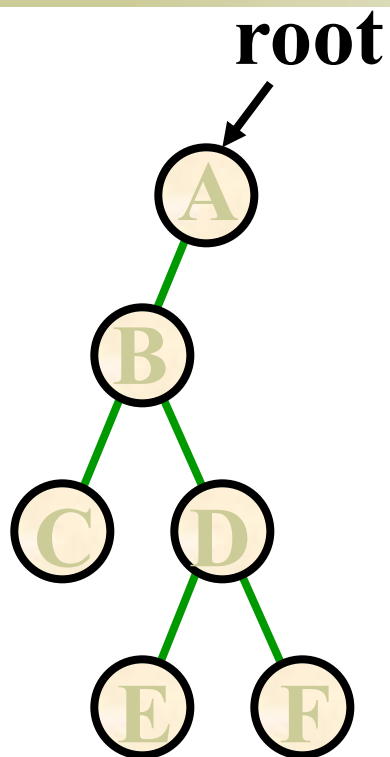


二叉链表



三叉链表

二叉树链表表示的示例



	data	parent	leftChild	rightChild
0	A	-1	1	-1
1	B	0	2	3
2	C	1	-1	-1
3	D	1	4	5
4	E	3	-1	-1
5	F	3	-1	-1

三叉链表的静态结构



二叉树的类定义



```
template <class T>
struct BinTreeNode {           //二叉树结点类定义
    T data;                   //数据域
    BinTreeNode<T> *leftChild, *rightChild;
                                //左子女、右子女链域
    BinTreeNode ()            //构造函数
    { leftChild = NULL; rightChild = NULL; }
    BinTreeNode (T x, BinTreeNode<T> *l = NULL,
                  BinTreeNode<T> *r = NULL)
    { data = x; leftChild = l; rightChild = r; }
};
```



```
template <class T>
```

```
class BinaryTree {
```

//二叉树类定义

```
public:
```

```
    BinaryTree () : root (NULL) { }        //构造函数
```

```
    BinaryTree (T value) : RefValue(value), root(NULL) { }
```

//构造函数

```
    BinaryTree (BinaryTree<T>& s);        //复制构造函数
```

```
    ~BinaryTree () { destroy(root); }      //析构函数
```

```
    bool IsEmpty () { return root == NULL; }
```

//判二叉树空否

```
    int Height () { return Height(root); } //求树高度
```

```
    int Size () { return Size(root); }     //求结点数
```



```
BinTreeNode<T> *Parent (BinTreeNode <T> *t)
```

```
{ return (root == NULL || root == t) ?
```

```
    NULL : Parent (root, t); } //返回双亲结点
```

```
BinTreeNode<T> *LeftChild (BinTreeNode<T> *t)
```

```
{ return (t != NULL)? t->leftChild : NULL; }
```

```
    //返回左子女
```

```
BinTreeNode<T> *RightChild (BinTreeNode<T> *t)
```

```
{ return (t != NULL)? t->rightChild : NULL; }
```

```
    //返回右子女
```

```
BinTreeNode<T> *getRoot () const { return root; }
```

```
    //取根
```



```
void preOrder (void (*visit) (BinTreeNode<T> *t))
    { preOrder (root, visit); }           //前序遍历
void inOrder (void (*visit) (BinTreeNode<T> *t))
    { inOrder (root, visit); }           //中序遍历
void postOrder (void (*visit) (BinTreeNode<T> *t))
    { postOrder (root, visit); }         //后序遍历
void levelOrder (void (*visit)(BinTreeNode<T> *t));
                                           //层次序遍历
int Insert (const T item);               //插入新元素
BinTreeNode<T> *Find (T item) const;     //搜索
```



protected:

```
BinTreeNode<T> *root; //二叉树的根指针
T RefValue;           //数据输入停止标志
void CreateBinTree (istream& in,
                    BinTreeNode<T> *& subTree);
                    //从文件读入建树
bool Insert (BinTreeNode<T> *& subTree, T& x);
                    //插入
void destroy (BinTreeNode<T> *& subTree);
                    //删除
bool Find (BinTreeNode<T> *subTree, T& x);
                    //查找
```



```
BinTreeNode<T> *Copy (BinTreeNode<T> *r);
```

//复制

```
int Height (BinTreeNode<T> *subTree);
```

//返回树高度

```
int Size (BinTreeNode<T> *subTree);
```

//返回结点数

```
BinTreeNode<T> *Parent (BinTreeNode<T> *  
subTree, BinTreeNode<T> *t);
```

//返回父结点

```
BinTreeNode<T> *Find (BinTreeNode<T> *  
subTree, T& x) const;    //搜寻x
```



```
void Traverse (BinTreeNode<T> *subTree,  
               ostream& out);           //遍历输出  
void preOrder (BinTreeNode<T>& subTree,  
               void (*visit) (BinTreeNode<T> *t));  
                               //前序遍历  
void inOrder (BinTreeNode<T>& subTree,  
              void (*visit) (BinTreeNode<T> *t));  
                               //中序遍历  
void postOrder (BinTreeNode<T>& Tree,  
                void (*visit) (BinTreeNode<T> *t));  
                               //后序遍历
```




```
friend istream& operator >> (istream& in,  
    BinaryTree<T>& Tree); //重载操作: 输入  
friend ostream& operator << (ostream& out,  
    BinaryTree<T>& Tree); //重载操作: 输出  
};
```

部分成员函数的实现

```
template <class T>  
BinTreeNode<T> *BinaryTree<T>::  
Parent(BinTreeNode <T> *subTree,  
    BinTreeNode <T> *t) {
```



```
//私有函数: 从结点 subTree 开始, 搜索结点 t 的双亲,  
//若找到则返回双亲结点地址, 否则返回NULL  
    if (subTree == NULL) return NULL;  
    if (subTree->leftChild == t || subTree->rightChild == t )  
        return subTree;           //找到, 返回父结点地址  
    BinTreeNode <T> *p;  
    if ((p = Parent (subTree->leftChild, t)) != NULL)  
        return p;                 //递归在左子树中搜索  
    else return Parent (subTree->rightChild, t);  
                                   //递归在右子树中搜索  
};
```



```
template<class T>
```

```
void BinaryTree<T>::
```

```
destroy (BinTreeNode<T> * subTree) {
```

```
//私有函数: 删除根为subTree的子树
```

```
    if (subTree != NULL) {
```

```
        destroy (subTree->leftChild);
```

```
//删除左子树
```

```
        destroy (subTree->rightChild);
```

```
//删除右子树
```

```
        delete subTree;
```

```
//删除根结点
```

```
    }
```

```
};
```



5.4.1 二叉树遍历的递归算法

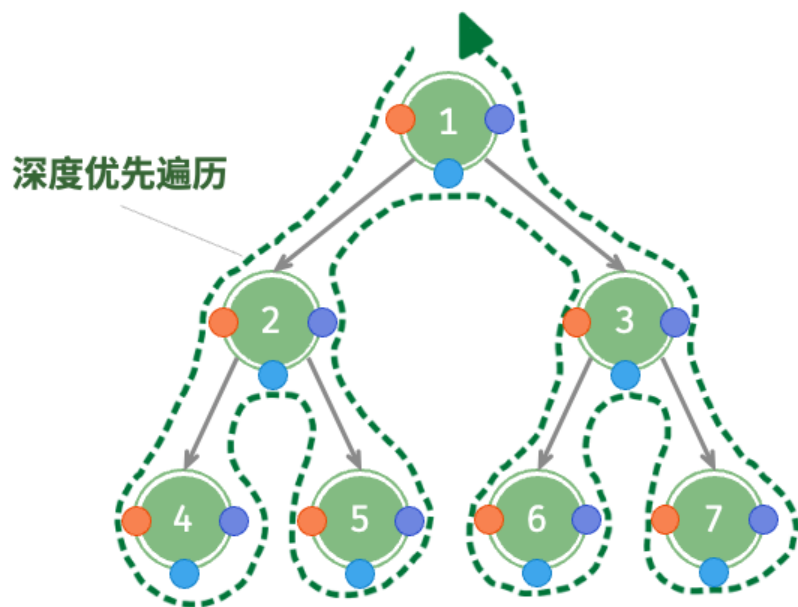


- 二叉树的遍历就是按某种次序访问树中的结点，要求每个结点访问一次且仅访问一次。
- 设 访问根结点记作 V
遍历根的左子树记作 L
遍历根的右子树记作 R
- 则可能的遍历次序有（先左后右+镜像）

PreOrder	前序	VLR	镜像	VRL
InOrder	中序	LVR	镜像	RVL
PostOrder	后序	LRV	镜像	RLV



前序、中序、后序遍历



```
/* 二叉树深度优先遍历 */  
void dfs(TreeNode root) {  
    if (root == null) return;  
    ● 即将访问左子树  
    dfs(root.left); // 访问左子树  
    ● 已访问左子树, 即将访问右子树  
    dfs(root.right); // 访问右子树  
    ● 左右子树都已访问, 即将回溯  
}
```

前序遍历 (节点在 ● 处被访问)

1, 2, 4, 5, 3, 6, 7

中序遍历 (节点在 ● 处被访问)

4, 2, 5, 1, 6, 3, 7

后序遍历 (节点在 ● 处被访问)

4, 5, 2, 6, 7, 3, 1



中序遍历 (Inorder Traversal)

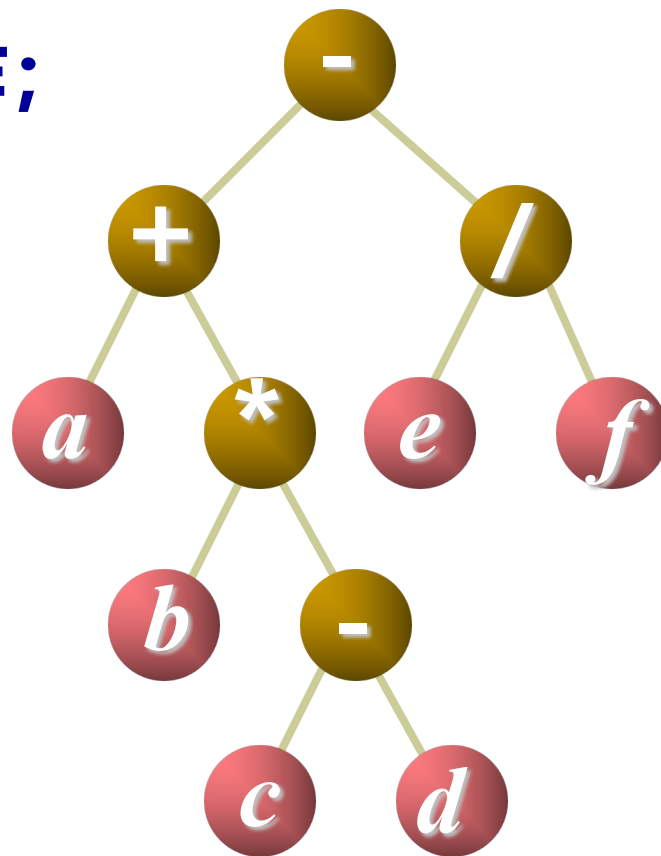


中序遍历二叉树算法的框架是：

- 若二叉树为空，则空操作；
- 否则
 - ◆ 中序遍历左子树 (L)；
 - ◆ 访问根结点 (V)；
 - ◆ 中序遍历右子树 (R)。

遍历结果

$$a + b * c - d - e / f$$





二叉树递归的中序遍历算法



```
template <class T>
void BinaryTree<T>::InOrder (BinTreeNode<T> *
    subTree, void (*visit) (BinTreeNode<T> *t)) {
    if (subTree != NULL) {
        InOrder (subTree->leftChild, visit);
        //遍历左子树
        visit (subTree);
        //访问根结点
        InOrder (subTree->rightChild, visit);
        //遍历右子树
    }
};
```



前序遍历 (Preorder Traversal)

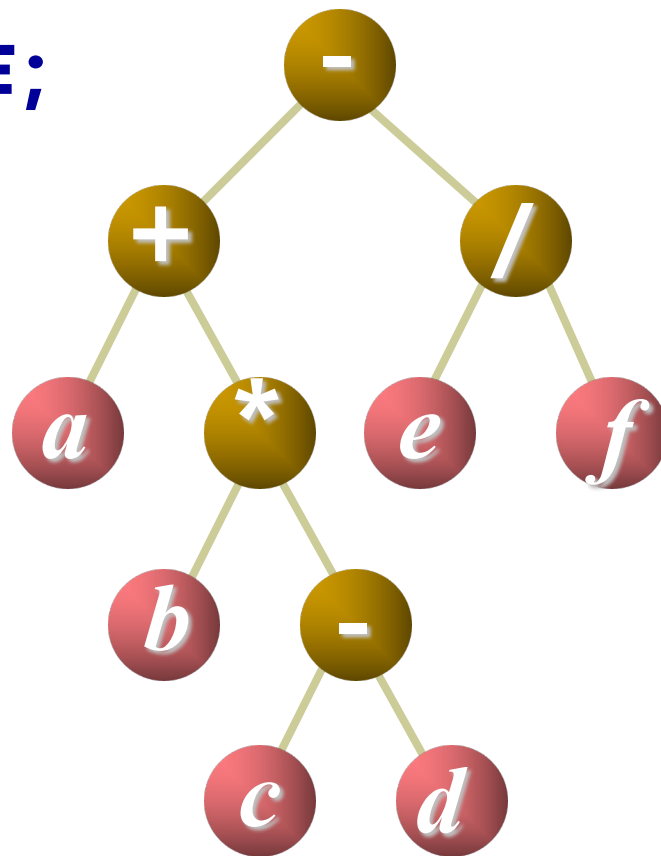


前序遍历二叉树算法的框架是：

- 若二叉树为空，则空操作；
- 否则
 - ◆ 访问根结点 (V)；
 - ◆ 前序遍历左子树 (L)；
 - ◆ 前序遍历右子树 (R)。

遍历结果

$- + a * b - c d / e f$





二叉树递归的前序遍历算法



```
template <class T>
void BinaryTree<T>::PreOrder (BinTreeNode<T> *
    subTree, void (*visit) (BinTreeNode<T> *t)) {
    if (subTree != NULL) {
        visit (subTree);           //访问根结点
        PreOrder (subTree->leftChild, visit);
                                   //遍历左子树
        PreOrder (subTree->rightChild, visit);
                                   //遍历右子树
    }
};
```



后序遍历 (Postorder Traversal)

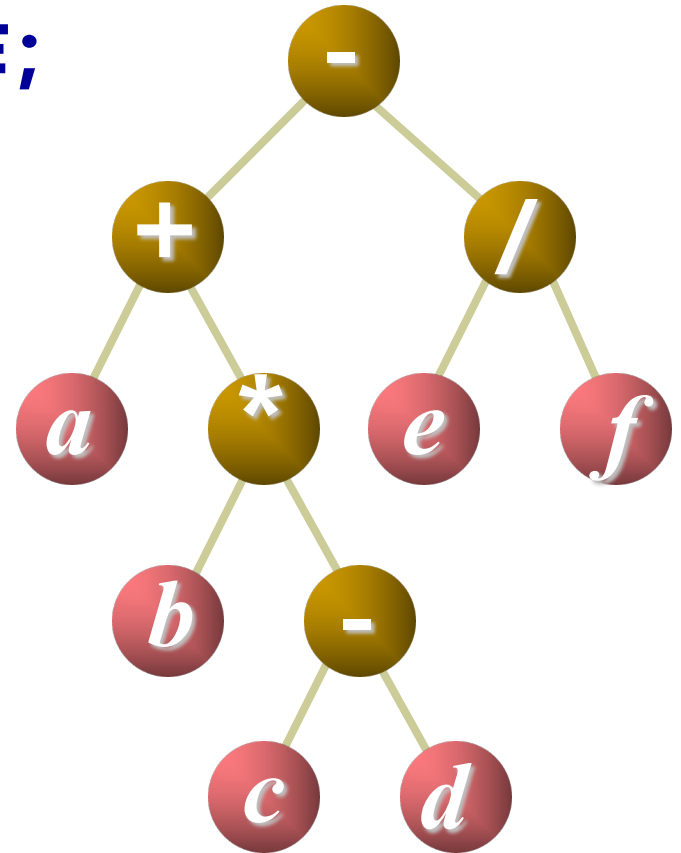


后序遍历二叉树算法的框架是：

- 若二叉树为空，则空操作；
- 否则
 - ◆ 后序遍历左子树 (L)；
 - ◆ 后序遍历右子树 (R)；
 - ◆ 访问根结点 (V)。

遍历结果

$a b c d - * + e f / -$





二叉树递归的后序遍历算法



```
template <class T>
void BinaryTree<T>::PostOrder (BinTreeNode<T> *
    subTree, void (*visit) (BinTreeNode<T> *t) ) {
    if (subTree != NULL ) {
        PostOrder (subTree->leftChild, visit);
        //遍历左子树
        PostOrder (subTree->rightChild, visit);
        //遍历右子树
        visit (subTree);
        //访问根结点
    }
};
```



5.4.2 二叉树遍历的应用



```
template <class T>
int BinaryTree<T>::Size (BinTreeNode<T> *
    subTree) const {
//私有函数：利用二叉树后序遍历算法计算二叉树
//的结点个数
    if (subTree == NULL) return 0;          //空树
    else return 1+Size (subTree->leftChild)
        + Size (subTree->rightChild);
};
```



5.4.2 二叉树遍历的应用



```
template <class T>
```

```
int BinaryTree<T>::Height ( BinTreeNode<T> *  
    subTree) const {
```

```
//私有函数： 利用二叉树后序遍历算法计算二叉树  
//的高度或深度
```

```
    if (subTree == NULL) return 0; //空树高度为0
```

```
    else {
```

```
        int i = Height (subTree->leftChild);
```

```
        int j = Height (subTree->rightChild);
```

```
        return (i < j) ? j+1 : i+1;
```

```
    };
```



利用二叉树前序遍历建立二叉树



- 以递归方式建立二叉树。
- 输入结点值的顺序必须对应二叉树结点前序遍历的顺序。
- 并约定以输入序列中不可能出现的值作为空结点的值以结束递归, 此值在RefValue中。例如用 “@”或用 “-1”表示字符序列或正整数序列空结点。

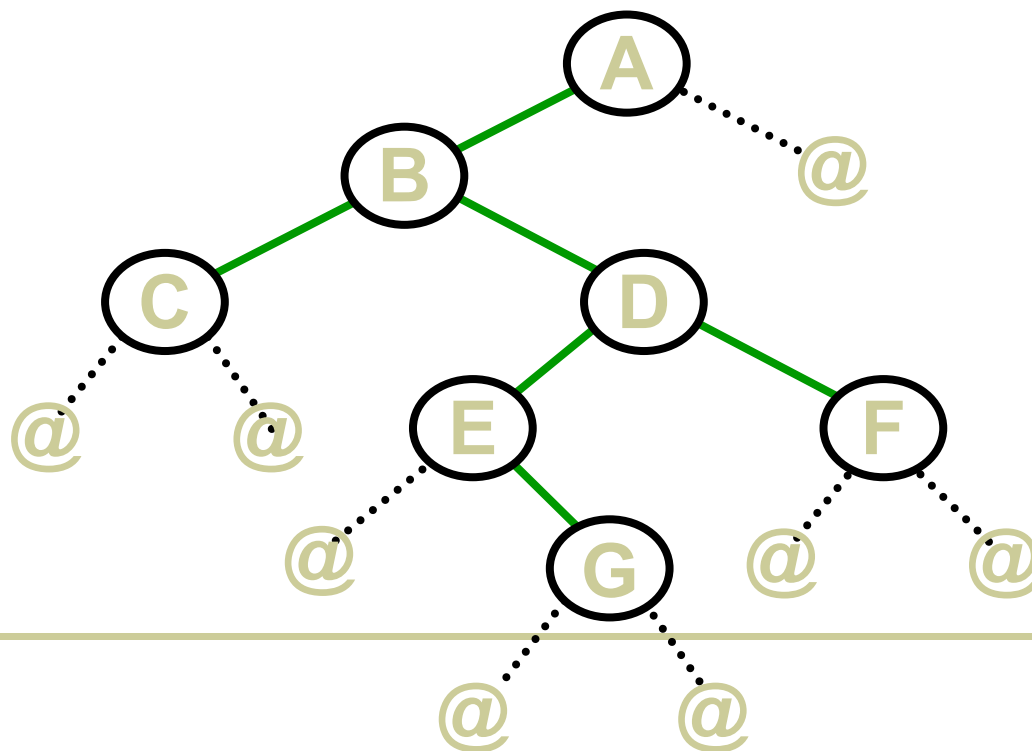


利用二叉树前序遍历建立二叉树



如图所示的二叉树的前序遍历顺序为

A B C @ @ D E @ G @ @ F @ @ @





利用二叉树前序遍历建立二叉树



```
template<class T>
```

```
void BinaryTree<T>::CreateBinTree (ifstream& in,  
    BinTreeNode<T> *& subTree) {
```

```
//私有函数: 以递归方式建立二叉树。
```

```
    T item;
```

```
    if ( !in.eof () ) {           //未读完, 读入并建树
```

```
        in >> item;               //读入根结点的值
```

```
        if (item != RefValue) {
```

```
            subTree = new BinTreeNode<T>(item);
```

```
            //建立根结点
```

```
            if (subTree == NULL)
```

```
                {cerr << “存储分配错!” << endl; exit (1);} }
```




利用二叉树前序遍历建立二叉树



```
CreateBinTree (in, subTree->leftChild);  
//递归建立左子树
```

```
CreateBinTree (in, subTree->rightChild);  
//递归建立右子树
```

```
}
```

```
else subTree = NULL;
```

```
//封闭指向空子树的指针
```

```
}
```

```
};
```